Vulnerability Assessment Report

Imagur Vulnerability Assessment Report

Imagur
132 Morphett Rd
Glengowrie SA 5044, Australia

Flinders University
Sturt Road, Bedford Park SA 5042

September 18, 2026

Report Version: 1.0

CONFIDENTIAL

Table of Contents

List of Figures 4

1 Document Control 5

1.1 Team 5

1.2 List of Changes 5

2 Executive Summary 6

2.1 Overview 6

2.2 Identified Vulnerabilities 7

3 Methodology 9

3.1 Objective 9

3.2 Scope 9

3.3 User Accounts and Permissions 10

4 Findings 11

I1: AWS access keys published in public JavaScript 11

I2: CloudFront hostname uses the wrong TLS certificate 12

I3: Extra Apache site on port 8080 13

I4: HTTPS directories redirect to HTTP 14

I5: No CSRF protection on account and upload actions 15

I6: PostgreSQL reachable on the public internet 16

I7: Public S3 bucket lists and serves private images 17

I8: SQL errors disclose database and source-file details 18

I9: SSH allows password logins from the internet 19

I10: Server and framework versions leaked 20

I11: Session cookie missing Secure, HttpOnly, and SameSite 21

I12: Stored cross-site scripting via image filename 22

I13: Upload form trusts client-supplied user_id (IDOR) 23

I14: www hostname certificate name mismatch 24

5 Disclaimer 25

A Appendix 26
A.1 Static Appendix Section 26
A.2 Coverage 26
A.3 Limitations 26
A.4 Methodology 26

List of Figures

Figure 1 - Distribution of identified vulnerabilities 8

1 Document Control

1.1 Team

Contact	Details	Role
ClawSec	Autonomous agent	Pentester

1.2 List of Changes

Version	Description	Date
1.0	Initial report of the authorized assessment completed on 18 September 2026.	Sep 18, 2026

2 Executive Summary

2.1 Overview

Imagur's public image-sharing site (Imager at imagesharer.xyz) was assessed on 18 September 2026 with the owner's authorization.

Imagur is a public image-sharing web application branded "Imager — your images, your control." The service runs PHP/nginx on imagesharer.xyz and offers login, register, dashboard, image upload (private/public), and profile avatars.

This assessment confirmed **14 findings: 2 critical, 2 high, 4 medium, 5 low, and 1 informational.**

The most serious issues are that customer image files sit in an Amazon S3 bucket that anyone can list and download — including images marked private — and that AWS keys used for that storage appear in a public JavaScript file. A logged-in user can also place an image into another customer's account, and a crafted filename can run script in visitors' browsers. Session cookies and forms are weakly protected, and the database and extra web ports are visible on the internet.

Highest-priority actions

1. Lock the S3 bucket.
2. Rotate and remove the published AWS keys.
3. Bind uploads to the logged-in user on the server.
4. Encode filenames.

Findings at a glance

Severity	Title
Critical	Public S3 bucket lists and serves private images
Critical	AWS access keys published in public JavaScript
High	Upload form trusts client-supplied user_id (IDOR)
High	Stored cross-site scripting via image filename
Medium	Session cookie missing Secure, HttpOnly, and SameSite
Medium	No CSRF protection on account and upload actions
Medium	PostgreSQL reachable on the public internet
Medium	SQL errors disclose database and source-file details
Low	SSH allows password logins from the internet
Low	Extra Apache site on port 8080
Low	HTTPS directories redirect to HTTP
Low	www hostname certificate name mismatch
Low	CloudFront hostname uses the wrong TLS certificate

Severity	Title
Info	Server and framework versions leaked

2.2 Identified Vulnerabilities

#	CVSS	Description	Page
I1	0.0	AWS access keys published in public JavaScript	11
I2	0.0	CloudFront hostname uses the wrong TLS certificate	12
I3	0.0	Extra Apache site on port 8080	13
I4	0.0	HTTPS directories redirect to HTTP	14
I5	0.0	No CSRF protection on account and upload actions	15
I6	0.0	PostgreSQL reachable on the public internet	16
I7	0.0	Public S3 bucket lists and serves private images	17
I8	0.0	SQL errors disclose database and source-file details	18
I9	0.0	SSH allows password logins from the internet	19
I10	0.0	Server and framework versions leaked	20
I11	0.0	Session cookie missing Secure, HttpOnly, and SameSite	21
I12	0.0	Stored cross-site scripting via image filename	22
I13	0.0	Upload form trusts client-supplied user_id (IDOR)	23
I14	0.0	www hostname certificate name mismatch	24

Vulnerability Overview

In the course of this penetration test **14 Info** vulnerabilities were identified:

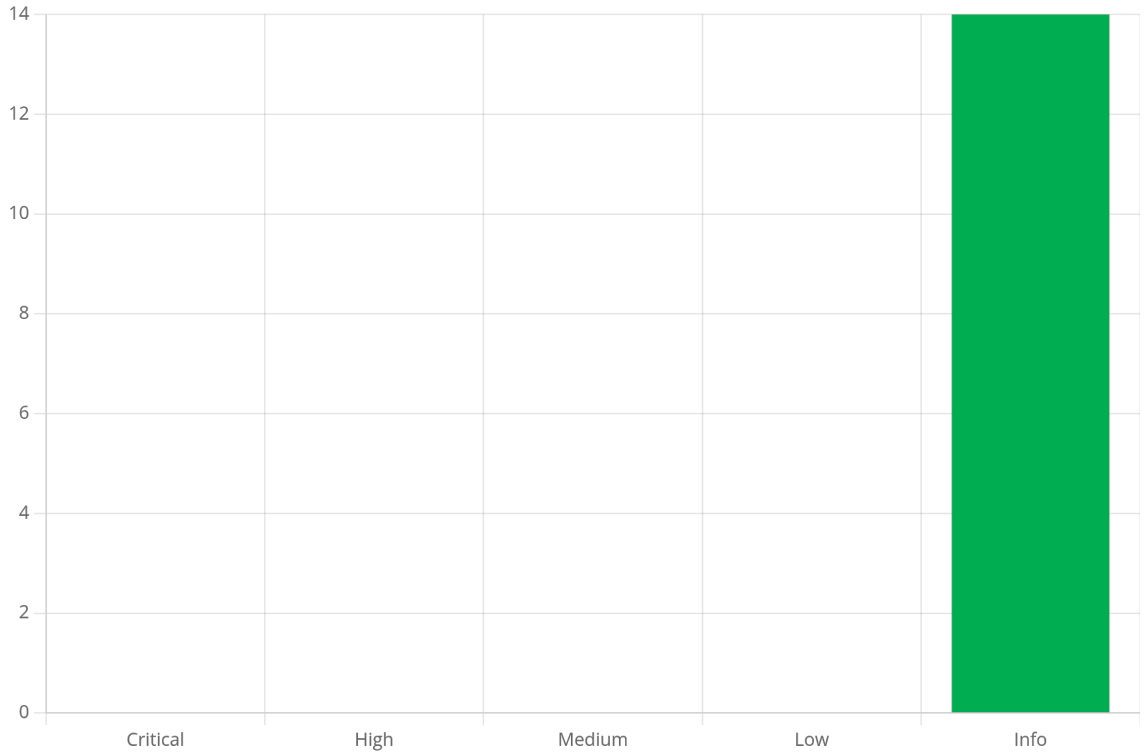


Figure 1 - Distribution of identified vulnerabilities

3 Methodology

This entire vulnerability assessment process was completed using agentic AI and existing LLM models. The purpose of such assessments is to automatically

3.1 Objective

The entire objective of this research is to evaluate the effectiveness of autonomously evaluate security posture for any business available on Google Map without the needs of technical guidance to determine how feasiable, seamless and cost-affective such approach is.

3.2 Scope

Authorized, non-destructive full-scope web and origin assessment of Imagur (public brand: Imager). Testing was limited to owner-approved assets. Neighboring cloud customers and lookalike domains were excluded.

In-scope (owner-approved)

System	Description
imagesharer.xyz (http and https)	Main Imager website
www.imagesharer.xyz	Same site as the main website
resources.imagesharer.xyz	Imagur CloudFront hostname
d273uevi0662l0.cloudfront.net	CloudFront distribution hostname for this Imagur distribution only
51.161.131.44	Origin IP only as the host of the website (not neighboring OVH customers)
S3 bucket filesharer-research	Discovered through the in-scope upload feature; tested only as Imagur image storage. Other customers' objects were not downloaded

Out of scope

System	Description
imagur.xyz, imager.xyz, imagesharer.net, fshare.xyz	Lookalike / related domains excluded after owner scope choice
imgur.com, fshare.vn	Unrelated third-party brands
Flinders University	Third party
Google	Third party
Spaceship/Cloudflare shared DNS	Shared platforms

System	Description
OVH netblock except 51.161.131.44	Other customers on the same provider
CloudFront co-tenants	Shared CDN platform
AWS account APIs	No use of leaked AWS secrets against the AWS account APIs

Methodology

Authorized, non-destructive full-scope web and origin assessment. Fast port probe then version scan of open ports only. Authenticated testing via public self-registration of two assessment accounts. Manual verification of upload, IDOR, XSS, storage, cookies, and TLS. No DDoS, malware, phishing, credential stuffing, password spraying, brute force, or destructive techniques.

3.3 User Accounts and Permissions

No production customer accounts were supplied by the owner.

Authenticated testing used two assessment accounts created through the public self-registration page on imagesharer.xyz.

Password authentication on SSH and PostgreSQL was not brute-forced.

4 Findings

I1: AWS access keys published in public JavaScript	
Score	0.0 (Info)
Vector string	N/A
Target	https://imagesharer.xyz/js/awsvalid.js
References	https://owasp.org/www-community/vulnerabilities/Use_of_hard-coded_cryptographic_key

Overview

A publicly downloadable script on the website contains live-looking AWS access-key material that matches the key id used to sign customer image URLs.

Details

https://imagesharer.xyz/js/awsvalid.js is reachable without login (HTTP 200, text/javascript). It implements AWS Signature Version 4 and STS GetCallerIdentity. Comments in that file include access key id AKIAQHQ6PGM62T2FFPGU and a secret access key. The same access key id appears in X-Amz-Credential on production S3 URLs issued by the application for uploaded images. The secret itself is not repeated here. Anyone who fetches the script can impersonate this AWS principal until the key is rotated. Business impact: attackers may read or change cloud data, run up billing, or use the account for further abuse.

Recommendation

Disable and rotate the exposed access key immediately in IAM. Remove awsvalid.js from the public site. Store credentials only in server-side secrets, not in browser code. Search the account for other long-lived keys and enable CloudTrail alerts on root/IAM key use.

I2: CloudFront hostname uses the wrong TLS certificate	
Score	0.0 (Info)
Vector string	N/A
Target	https://resources.imagesharer.xyz/
References	-

Overview

The resources hostname does not present a certificate for resources.imagesharer.xyz.

Details

Strict TLS to https://resources.imagesharer.xyz/ fails (certificate subject mismatch; CloudFront default *.cloudfront.net). The distribution d273uevi0662l0.cloudfront.net is the CNAME target. With verification skipped, requests returned CloudFront 403; icon.ico was not readable. Business impact: browsers will not trust this hostname; leftover references cannot load securely.

Recommendation

Attach a matching certificate (ACM) for resources.imagesharer.xyz, or stop using that hostname. Do not leave commented production asset hosts misconfigured.

I3: Extra Apache site on port 8080	
Score	0.0 (Info)
Vector string	N/A
Target	• 51.161.131.44:8080 • http://imagesharer.xyz:8080/
References	-

Overview

The same PHP app is also exposed on an extra Apache port, increasing the attack surface.

Details

Port 8080 serves Apache/2.4.68 (Debian) with PHP/8.2.33 and redirects to /dashboard, same Imager app as nginx on 80/443. HTTPS is not offered on 8080. Business impact: a second, less-polished entry point if nginx rules and Apache rules differ.

Recommendation

Close 8080 on the firewall unless required. If needed internally, bind to localhost.

I4: HTTPS directories redirect to HTTP	
Score	0.0 (Info)
Vector string	N/A
Target	• https://imagesharer.xyz/css • https://imagesharer.xyz/js • https://imagesharer.xyz/uploads
References	-

Overview

Some HTTPS URLs send the browser to HTTP before sending them back to HTTPS.

Details

GET https://imagesharer.xyz/css returned 301 Location: http://imagesharer.xyz/css/. The HTTP URL then 301s back to HTTPS. Directory URLs /js and /uploads behaved the same. This mixed redirect can expose non-Secure cookies on the HTTP hop. Business impact: session or request data may be visible on the network.

Recommendation

Redirect only to HTTPS. Enable HSTS. Fix trailing-slash redirects to stay on HTTPS.

I5: No CSRF protection on account and upload actions	
Score	0.0 (Info)
Vector string	N/A
Target	• https://imagesharer.xyz/register • https://imagesharer.xyz/login • https://imagesharer.xyz/images/upload • https://imagesharer.xyz/profile/avatar • https://imagesharer.xyz/logout
References	https://owasp.org/www-community/attacks/csrf

Overview

Important forms have no anti-CSRF token, and the session cookie lacks SameSite, so other websites can try to submit those forms as a logged-in user.

Details

Login, register, image upload, avatar upload, and logout forms had no hidden CSRF token. Registration succeeded with a simple POST of email and password. Upload and avatar accept multipart POST with only the visible fields. PHPSESSID has no SameSite flag. Business impact: a malicious page could change a signed-in customer's avatar, upload files, or log them out.

Recommendation

Add per-session CSRF tokens and verify them server-side. Set SameSite on the session cookie. Require re-authentication for sensitive changes.

I6: PostgreSQL reachable on the public internet	
Score	0.0 (Info)
Vector string	N/A
Target	51.161.131.44:5432
References	https://www.postgresql.org/docs/current/auth-pg-hba-conf.html

Overview

The database port on the website server accepts connections from the internet.

Details

Port scan of the approved origin 51.161.131.44 showed 5432/tcp open. The service speaks PostgreSQL protocol 3. A single unauthenticated client attempt was rejected with "fe_sendauth: no password supplied" — password authentication is required. No passwords were guessed. The port still should not be public. Verbose application SQL errors (see related finding) confirm PostgreSQL is used by Imager. Business impact: attackers can keep trying to log in to the database from anywhere if a password is weak or later leaked.

Recommendation

Bind PostgreSQL to localhost or a private network. Restrict with firewall/security group to 51.161.131.44:5432 from nowhere on the internet. Require strong credentials and TLS.

I7: Public S3 bucket lists and serves private images	
Score	0.0 (Info)
Vector string	N/A
Target	• filesharer-research.s3.ap-southeast-2.amazonaws.com • https://imagesharer.xyz/ (image storage)
References	• https://owasp.org/www-community/vulnerabilities/Insecure_Storage • https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-control-block-public-access.html

Overview

Anyone on the internet can list files in Imagur's image bucket and download them, including images marked Private.

Details

Uploaded images are stored in Amazon S3 bucket filesharer-research in region ap-southeast-2, under predictable keys users//.png. An unauthenticated GET of the bucket root returned ListBucket XML (HTTP 200), including objects belonging to more than one user. An unauthenticated GET of our own test object returned HTTP 200 and the PNG bytes, with no signature required. The same unsigned GET succeeded for an image uploaded with is_public=false. The website correctly sent unauthenticated browsers away from the private image page, but the file is still readable on S3. Business impact: customer photos that users believe are private can be discovered and downloaded by anyone who knows or lists the bucket.

Recommendation

Turn on S3 Block Public Access. Make the bucket private. Serve files only through short-lived presigned URLs or a backend proxy. Disable listing. Review existing objects for exposure and rotate any affected links.

I8: SQL errors disclose database and source-file details	
Score	0.0 (Info)
Vector string	N/A
Target	https://imagesharer.xyz/images/upload
References	https://owasp.org/www-community/Improper_Error_Handling

Overview

Invalid input returns a raw PostgreSQL error and an internal PHP file path.

Details

POST /images/upload with user_id equal to a valid UUID plus a single quote returned HTTP 500. The body included SQLSTATE[22P02] (invalid UUID syntax) and CONTEXT pointing at ImagesService.php:71. This is error disclosure, not a demonstrated SQL injection. Business impact: helps attackers map internals and tune further attacks.

Recommendation

Show a generic error to users. Log details server-side only. Disable debug in production.

I9: SSH allows password logins from the internet	
Score	0.0 (Info)
Vector string	N/A
Target	51.161.131.44:22
References	-

Overview

The server's remote-admin service accepts password authentication from the internet.

Details

OpenSSH 10.0p2 is on 51.161.131.44:22. ssh-auth-methods reported publickey and password. Algorithms observed were modern. No login attempts were made. Combined with a public IP, password SSH increases lockout/brute-force risk (not tested). Business impact: if an admin password is weak, the whole server can be taken over.

Recommendation

Disable PasswordAuthentication. Use keys only. Restrict SSH by firewall to admin IPs. Keep fail2ban or equivalent.

I10: Server and framework versions leaked	
Score	0.0 (Info)
Vector string	N/A
Target	• https://imagesharer.xyz/ • http://51.161.131.44:8080/
References	-

Overview

Responses tell the internet the exact web-server and PHP versions.

Details

nginx responses include X-Powered-By: PHP/8.2.33. Apache 403 pages include "Apache/2.4.68 (Debian) Server at imagesharer.xyz Port 80" even when the front door is nginx. Business impact: easier targeting of known bugs; no exploit of those versions was demonstrated here.

Recommendation

Hide X-Powered-By. Set Apache ServerTokens Prod and ServerSignature Off. Custom 403 pages.

I11: Session cookie missing Secure, HttpOnly, and SameSite	
Score	0.0 (Info)
Vector string	N/A
Target	https://imagesharer.xyz/ (PHPSESSID)
References	• https://owasp.org/www-community/controls/SecureCookieAttribute • https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie

Overview

The login session cookie can be sent over HTTP and read by script, and it is not restricted to same-site requests.

Details

Set-Cookie for PHPSESSID used Path=/ only. Flags Secure, HttpOnly, and SameSite were absent. A second cookie, page_views / last_login, did set Secure, HttpOnly, and SameSite=Lax. The site also issues HTTP 301 from https://imagesharer.xyz/css to http://imagesharer.xyz/css/, which is a path where a non-Secure session cookie would be transmitted in clear text. Business impact: session theft via XSS or network sniffing, and easier CSRF.

Recommendation

Set PHPSESSID with Secure; HttpOnly; SameSite=Lax (or Strict). Use HTTPS only. Do not redirect HTTPS requests to HTTP.

I12: Stored cross-site scripting via image filename	
Score	0.0 (Info)
Vector string	N/A
Target	• https://imagesharer.xyz/images/upload • https://imagesharer.xyz/dashboard • public image pages under /images/<id>
References	https://owasp.org/www-community/attacks/xss/

Overview

A crafted image filename is stored and later shown as HTML, which can run script in other people's browsers.

Details

Uploading a PNG whose filename contained ">

Recommendation

Treat filenames as untrusted text. HTML-encode on output. Restrict filename characters. Prefer displaying a server-generated name. Set HttpOnly on the session cookie. Consider a strict Content-Security-Policy.

I13: Upload form trusts client-supplied user_id (IDOR)	
Score	0.0 (Info)
Vector string	N/A
Target	https://imagesharer.xyz/images/upload
References	• https://owasp.org/www-community/attacks/Unvalidated_Redirects_and_Forwards • https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html

Overview

A logged-in user can attach an upload to someone else's account by changing a hidden user id.

Details

The authenticated upload form includes a hidden field user_id set to the current user's UUID. That value is accepted by POST /images/upload. Using assessment account A, we uploaded a benign 1x1 PNG with user_id set to assessment account B (25e88807-77e6-4363-a5c0-9790d7eeaa3f). The server returned 302 to a new image id. The image then appeared on B's dashboard and not on A's. No CSRF token is required. Business impact: an attacker can plant content in another customer's library, or clutter/abuse another account.

Recommendation

Ignore client-supplied user_id. Bind ownership only from the server session. Add authorization checks on create/update/delete. Add CSRF protection.

I14: www hostname certificate name mismatch	
Score	0.0 (Info)
Vector string	N/A
Target	https://www.imagesharer.xyz/
References	-

Overview

Browsers reject HTTPS for www because the certificate only names imagesharer.xyz.

Details

curl to https://www.imagesharer.xyz/login failed with subjectAltName not matching www.imagesharer.xyz. The Let's Encrypt certificate SAN is only imagesharer.xyz. DNS for www points at the same origin. Business impact: customers who type www see a certificate warning or cannot connect.

Recommendation

Reissue the certificate with both imagesharer.xyz and www.imagesharer.xyz, or redirect www on HTTP only after a valid cert exists.

5 Disclaimer

The purpose of this research project is to help evaluating the seamlessness, effectiveness and cost-benefit ratio of autonomously performing vulnerabilty assessment using agentic AI. I do not take any responsibility upon any misuse or abuse of such tool where it can face significant legal actions.

A Appendix

A.1 Static Appendix Section

A.2 Coverage

The following areas were covered during the 18 September 2026 assessment:

- Origin 51.161.131.44 ports 22, 80, 443, 5432, 8080
- Unauthenticated and authenticated Imager app (login, register, dashboard, profile, upload, avatars, JSON API)
- CloudFront resources hostname
- Application S3 storage for images

A.3 Limitations

- Password authentication on SSH/PostgreSQL was not brute-forced.
- Leaked AWS secret was not used to call AWS APIs; access-key id match to production S3 URLs was used as attribution.
- S3 listing showed other keys; those objects were not retrieved.
- Candidate lookalike domains were excluded after owner scope choice.
- No phone number on file.

A.4 Methodology

Authorized, non-destructive full-scope web and origin assessment.

- Fast port probe then version scan of open ports only.
- Authenticated testing via public self-registration of two assessment accounts.
- Manual verification of upload, IDOR, XSS, storage, cookies, TLS.
- No DDoS, malware, phishing, credential stuffing, password spraying, brute force, or destructive techniques.